HybriScan: A Category-Aware, Weighted Heuristic Web Vulnerability Scanner with Adaptive Threshold Optimisation

Reducing False Positives Through Structured Response-Content Analysis and Hard-Negative Discrimination

Dishant P. Suthar[1], Udbodh V. Solanki[2]
[1]Independent Researcher, Gujarat, India
[2]Independent Researcher, Gujarat, India
Correspondence: dishant08103@gmail.com[1], udbodhsolanki02@gmail.com[2]

Abstract

Automated web vulnerability scanners are a fundamental tool in the modern security practitioner's arsenal; however, traditional wordlist-based scanners suffer from elevated false-positive rates because they classify endpoints based solely on URL path matching without examining HTTP response content. This paper presents HybriScan, a lightweight, category-aware heuristic scanner that augments conventional path enumeration with a structured, weighted response-analysis engine. For each scanned endpoint, HybriScan computes a six-dimensional vulnerability score vector across administrative-panel, SQL-injection, authentication-interface, directory-listing, XSS-surface-indicator, and sensitive-file-exposure categories using precompiled regular expression pattern banks with empirically assigned weight coefficients. A global composite score is derived through inter-category maximum pooling, and endpoint classification is governed by an adaptive threshold optimised over successive scan iterations. We evaluate HybriScan against a controlled synthetic dataset of 27 endpoints spanning six semantic categories, including a hard-negative class of benign pages containing security-relevant terminology. HybriScan achieves an accuracy of 77.78%, a precision of 88.89%, and a false-positive rate (FPR) of 7.14%, compared to the pure-wordlist baseline which achieves 44.44% accuracy and 100% FPR. The F1 score improves from 0.6154 (baseline) to 0.7273 (HybriScan). We discuss per-category performance trade-offs, threshold sensitivity, and the limitations of the current implementation, and outline a concrete roadmap for real-world validation using DVWA and OWASP WebGoat environments.

Keywords: Web Application Security, Vulnerability Scanner, False Positive Reduction, Heuristic Pattern Detection, Adaptive Thresholding, Penetration Testing Automation, OWASP, Directory Enumeration.

Introduction

Web applications represent one of the most actively exploited attack surfaces in contemporary cybersecurity. The OWASP Top Ten [1] — updated in 2021 — continues to list injection flaws, authentication failures, and security misconfigurations among the most critical and prevalent vulnerabilities in deployed web software. Automated scanning tools are indispensable for identifying such weaknesses efficiently; however, the quality of their detection mechanisms directly determines their operational utility.

Production-grade tools such as Nikto [2], Gobuster [3], and Dirsearch [4] implement wordlist-based enumeration: they issue HTTP requests to a predetermined set of paths and examine status codes to classify endpoints. While computationally efficient, these tools suffer from a fundamental limitation: they classify endpoints based solely on path structure, without examining the content of HTTP responses. An HTTP 200 status code returned for any wordlist entry is flagged as a vulnerability, regardless of whether the response body contains any exploitable content. This approach produces false-positive rates that frequently reach 50-100% in real-world deployments [5], substantially degrading the signal-to-noise ratio of scan output and burdening security analysts with manual verification of benign findings.

In this paper, we present HybriScan — a heuristic hybrid scanner that addresses this limitation by integrating response-content analysis into the detection pipeline. Our approach does not require a trained machine learning model, an active network connection to a target server, or prior knowledge of the target application's framework. Instead, HybriScan

constructs a weighted, multi-category vulnerability score from the URL structure and the HTTP response body, enabling contextual discrimination that pure path-matching systems cannot achieve.

We make the following contributions:

- A six-category weighted heuristic scoring engine with 68 compiled regex patterns across administrative-panel, SQL-injection, authentication-interface, directory-listing, XSS-surface-indicator, and sensitive-file-exposure categories, enabling simultaneous detection of the most prevalent OWASP 2021 vulnerability classes.
- An adaptive threshold optimisation algorithm that iteratively adjusts the classification boundary to achieve a stable precision-recall operating point without model retraining.
- A structured evaluation methodology that explicitly includes hard-negative test cases — benign endpoints whose URL or content contains security-relevant keywords — as a discriminability stress test.
- A detailed per-category failure analysis that identifies the structural reasons for missed detections and false positives, providing a concrete foundation for targeted improvement.

The remainder of this paper is organised as follows. Section II reviews the related literature. Section III defines the threat model. Section IV describes the methodology. Section V presents the system architecture. Section VI details the implementation, including algorithm pseudocode. Section VII describes the experimental setup. Section VIII presents results. Section IX provides a discussion. Section X addresses ethical considerations. Section XI identifies limitations. Section XII concludes, and Section XIII outlines future work.

Literature Review

The detection of web application vulnerabilities through automated scanning has been studied for over two decades. We review the principal threads of related work that contextualise HybriScan's design.

A. Wordlist-Based Directory Enumeration

Directory enumeration scanners represent the baseline class of tools against which HybriScan is compared. Sullo and Lodge's Nikto (2002) [2] pioneered HTTP-level vulnerability scanning by combining directory brute-forcing with static signature checks against server responses. Gobuster [3] and Dirsearch [4] offer high-performance multi-threaded enumeration but remain purely wordlist-driven. Doupé et al. [6] conducted a systematic evaluation of black-box web scanners and demonstrated that the majority miss between 20% and 80% of known vulnerabilities when deployed against standardised benchmarks, while simultaneously generating large volumes of false positives. This evaluation motivates the need for response-content-aware detection mechanisms.

B. Response-Content Analysis in Scanning

Several researchers have proposed augmenting path-based scanning with HTTP response analysis. Bau et al. [7] described an architecture that augments standard crawling with application-state inference, enabling the scanner to distinguish authenticated from unauthenticated resource responses. Pellegrino et al. [8] proposed DOM-based analysis to identify reflected XSS sinks in JavaScript-rendered responses. These approaches demonstrate the principle that response content carries substantial discriminative information that status-code-only scanners discard. HybriScan implements a lightweight version of this principle using compiled regular expression pattern banks rather than DOM parsing or JavaScript execution.

C. Machine Learning in Vulnerability Detection

Machine learning has been applied to web vulnerability detection primarily in two contexts: payload generation and response classification. Paramitha and Kusrini [9] trained a naive Bayes classifier on HTTP request features to distinguish malicious from benign traffic with 87% accuracy. Shu et al. [10] applied transformer-based models to HTTP request sequences and reported significant improvements in detecting multi-step attacks. Li et al. [11] demonstrated that ML-based WAF evasion (WAF-A-MoLE) can defeat static rule-based defences, motivating the integration of adaptive mechanisms even in heuristic systems. However, ML-based approaches typically require labelled training corpora of thousands of examples and impose inference overhead incompatible with real-time scanning at scale. HybriScan is explicitly designed as an ML-free heuristic system, with the ML integration path described in Section XIII as future work.

D. False Positive Reduction

Reducing false positives is recognised as an open problem in automated security scanning. Bau et al. [7] found that the scanners they evaluated produced FPRs of 32-87% depending on the target environment. Doupé et al. [6] noted that false positive reduction requires semantic understanding of application responses rather than syntactic path matching. Commercial tools such as Burp Suite Professional [12] implement sophisticated response differencing to distinguish functionally significant responses from error pages; these approaches, while effective, are not available in open-source or lightweight deployment scenarios. HybriScan contributes an accessible, reproducible FPR reduction approach through weighted multi-category scoring.

E. Comparative Scanner Evaluations

Recent comparative studies have evaluated the performance of popular scanners against intentionally vulnerable benchmark applications. Testing against DVWA, WebGoat, and Mutillidae II has become a standard evaluation protocol [5], [6]. These benchmarks provide reproducible, legally accessible, ground-truth-labelled vulnerability sets that enable fair cross-tool comparison. We adopt this evaluation protocol as the standard for our proposed future validation (Section XIII), distinguishing the current controlled synthetic evaluation from the planned real-world assessment.

Threat Model

A. Adversary Profile

HybriScan is designed from the perspective of an authorised penetration tester or security auditor conducting a black-box assessment of a web application. The operator holds no credentials, has no access to application source code or database schemas, and has no prior knowledge of the target's internal structure beyond its base URL and general technology stack.

B. Target Environment

The target is a public-facing web application accessible over HTTP or HTTPS. The application may implement a web application firewall, rate limiting, or session-based authentication. HybriScan operates in the unauthenticated, non-session-aware mode and does not attempt to bypass authentication barriers.

C. Vulnerability Scope

HybriScan targets six vulnerability categories: (i) exposed administrative interfaces, accessible without authentication; (ii) SQL injection indicators, detectable through error message leakage; (iii) exposed authentication endpoints, which represent potential brute-force targets; (iv) misconfigured directory listings, which expose server file system structure; (v) XSS surface indicators, detectable through reflected input and DOM sink patterns in server responses; and (vi) sensitive file and path exposure, detectable through known sensitive paths and body content markers such as exposed environment configuration files. These categories correspond to OWASP Top Ten 2021 categories A01 (Broken Access Control), A03 (Injection), and A05 (Security Misconfiguration) [1].

D. Trust Boundaries

The scanner is assumed to operate from a network position with HTTP(S) access to the target. All traffic is generated by the scanner operator. HybriScan does not exfiltrate data, does not persist access, and does not attempt to exploit vulnerabilities — it performs detection only.

E. Out of Scope

HybriScan does not address: server-side request forgery (SSRF), XML external entity injection (XXE), authentication session management, JavaScript-rendered Single Page Applications (SPAs), or vulnerabilities requiring multi-step interaction sequences. These are identified as future extensions.

Methodology

A. Research Design

This study follows an experimental comparative design. A controlled dataset of web endpoints is constructed with ground-truth vulnerability labels. Both the baseline wordlist scanner and HybriScan are applied to the complete dataset,

and their detection outputs are evaluated using a binary classification framework. The evaluation produces confusion matrices, standard binary classification metrics, and per-category performance breakdowns. Results are reported without ex-post result engineering — the metrics reflect direct scanner output.

B. Dataset Construction

The evaluation dataset was synthetically generated to provide a controlled ground-truth reference. Six endpoint categories were constructed: (i) administrative panel endpoints with paths and simulated response bodies characteristic of admin interfaces; (ii) SQL injection targets with response bodies containing database error strings from MySQL, Oracle, and MSSQL dialects; (iii) login page endpoints with authentication form markup; (iv) directory listing endpoints with file-index HTML structures; (v) hard-negative endpoints — benign pages whose URL paths or response content contain security-relevant keywords (e.g., /blog/securing-admin-panels, /tutorial/login-systems) without containing actual vulnerability indicators; and (vi) safe endpoints representing ordinary application resources.

The current experimental dataset contains 27 endpoints (13 vulnerable, 14 benign). We explicitly acknowledge that this dataset size is insufficient for statistical significance testing and is appropriate only for a proof-of-concept evaluation. Section XIII describes the planned expansion to a 380-endpoint dataset using DVWA and WebGoat environments for real-world validation. Hard-negative endpoints are critical to the evaluation design: they test the scanner's capacity to avoid keyword-triggered false positives — a failure mode that afflicts pure path-matching systems.

C. Baseline Scanner

The baseline scanner implements a pure URL path-matching classifier: if any path segment matches a wordlist entry for admin, login, directory, or SQL-injectable paths, the endpoint is classified as vulnerable. No HTTP response content is examined. This replicates the classification logic of Nikto, Gobuster, and Dirsearch at the detection decision level and serves as the performance lower bound.

D. HybriScan Detection Pipeline

The HybriScan pipeline processes each endpoint through four sequential stages:

1. URL normalisation: The URL is parsed, the path is extracted and split into segments, and query parameters are isolated for separate analysis.
2. Response acquisition: The scanner issues asynchronous HTTP GET requests through the aiohttp client layer and retrieves the response body, headers, and metadata for downstream analysis. For the controlled experimental evaluation, synthetic response bodies were additionally used to ensure deterministic category coverage across all dataset endpoints.
3. Category-specific pattern scoring: The URL and response body are evaluated against six independent pattern banks — covering administrative interfaces, SQL injection indicators, authentication endpoints, directory listings, XSS surface indicators, and sensitive file exposure. Each pattern bank contains compiled regular expressions with associated weight coefficients. For each pattern match, the weight is accumulated into a raw category score. The raw score is normalised to [0, 1] by dividing by the maximum achievable score for that category.
4. Threshold classification: A composite score is derived as the maximum normalised category score. If the composite score exceeds the adaptive threshold T, the endpoint is classified as vulnerable; otherwise, it is classified as benign.

E. Adaptive Threshold Optimisation

The classification threshold T is initialised to 0.70 and refined over successive evaluation iterations. At each iteration, if the observed FPR exceeds a target FPR of 0.15, T is incremented by a fixed step of 0.01 to tighten the classification boundary. If accuracy falls below 0.75, T is decremented by a fixed step of 0.01 to recover recall. This process terminates when the operating point satisfies the convergence criteria or the maximum iteration count is reached: T is bounded within [0.50, 0.82].

Important note on methodology: The threshold optimisation is applied to the training set (the evaluation dataset itself) and is therefore a form of threshold selection on in-sample data. This is analogous to threshold tuning on a validation set without a separate held-out test set. Reported metrics reflect in-sample performance. The planned real-world evaluation described in Section XIII will implement a proper train/validation/test split to produce out-of-sample performance estimates.

F. Evaluation Metrics

Binary classification performance is evaluated using: accuracy = (TP + TN) / N; precision = TP / (TP + FP); recall (TPR) = TP / (TP + FN); false-positive rate (FPR) = FP / (FP + TN); and F1 score = 2 × (precision × recall) / (precision + recall). These metrics are derived from the complete confusion matrix at the selected operating threshold.

System Architecture

HybriScan is structured as a five-layer modular pipeline. The architectural principle is strict layer separation: each layer has a single responsibility, communicates through well-defined data structures, and can be replaced or extended independently.

A. Input and URL Normalisation Layer

The input layer receives a target base URL and a wordlist configuration. It constructs fully-qualified endpoint URLs by appending each wordlist entry to the base URL. The normaliser extracts the URL path, splits it into path segments, isolates query string parameters, and passes a structured URL object to the response acquisition layer. This normalisation step ensures that equivalent URLs with different formatting (trailing slashes, URL encoding) are treated consistently.

B. Response Acquisition Layer

This layer issues asynchronous HTTP GET requests using the aiohttp library within a shared ClientSession, enforces a configurable per-request timeout, applies exponential back-off retry logic up to a configurable maximum, follows redirects up to a configurable maximum depth, and returns the response status code, lower-cased headers, and decoded body as a structured ScanResult dataclass. Concurrency is bounded by an asyncio.Semaphore set to the configured limit (default: 10 concurrent requests). The response acquisition layer abstracts the transport mechanism from the detection engine, ensuring that the pattern scoring logic is independent of the connection management strategy.

C. Pattern Detection Engine

The pattern detection engine is the core technical contribution of HybriScan. It maintains six independent category pattern banks covering: administrative interface exposure, SQL injection indicators, authentication endpoint detection, directory listing misconfiguration, XSS surface indicators, and sensitive file exposure. Each bank contains a list of (compiled_regex, weight) tuples. For a given URL and response body, the engine iterates over each pattern bank, counts the number of pattern matches in the concatenated URL+body string, multiplies each match count by the pattern's weight, and accumulates a raw category score S_raw. The normalised category score S_norm = min(1.0, S_raw / S_max), where S_max is the theoretical maximum raw score for the category.

D. Scoring and Classification Layer

The scoring layer receives six normalised category scores [S_admin, S_sqli, S_login, S_dir, S_xss, S_sensitive]. The composite vulnerability score is derived as: V = max(S_admin, S_sqli, S_login, S_dir, S_xss, S_sensitive). The endpoint is classified as vulnerable if V ≥ T (the adaptive threshold), and benign otherwise. The score vector is preserved in the output record for diagnostic and visualisation purposes.

E. Output and Reporting Layer

Each processed endpoint produces a structured result record containing: the endpoint URL, predicted label, composite score V, six-dimensional category score vector, dominant vulnerability category label, per-endpoint evidence list, security header audit, and processing time in milliseconds. All records are serialised to a timestamped JSON report file.

Implementation

A. Technology Stack

HybriScan is implemented in Python 3.12+ using the following standard and third-party libraries: re (compiled regular expression engine with IGNORECASE and MULTILINE flags); json and PyYAML 6.0+ (structured report serialisation and configuration management); aiohttp 3.9+ (asynchronous HTTP client with session management and exponential back-off retry); BeautifulSoup4 4.12+ with lxml parser (HTML response parsing for form, title, and script analysis);

asyncio (concurrent request scheduling via Semaphore); matplotlib 3.8+ and numpy 1.26+ (visualisation and numeric operations); and Rich 13.7+ (terminal output formatting). The complete implementation is available as an open-source Python package comprising nine core modules, a CLI entry point (main.py), and comprehensive modular test suite, enabling immediate reproducibility with pip install -r requirements.txt.

B. Pattern Bank Design

Table I summarises the pattern bank structure for each detection category.

TABLE I. HybriScan Pattern Bank Summary

Category	Pattern Count	Weight Range	Example High-Weight Pattern
Admin Detection	15	0.20 – 0.95	r'wp-admin|/administrator' in URL (w=0.95)
SQL Injection	14	0.55 – 0.95	r'You have an error in your SQL syntax' (w=0.95)
Login Detection	11	0.60 – 0.90	<input type="password"> in body (w=0.90)
Directory Listing	8	0.75 – 1.00	r'Index of /' in body title (w=1.00)
XSS Indicators	9	0.50 – 0.80	HTML tag injected in query parameter (w=0.80)
Sensitive File Exposure	11	0.80 – 0.95	DB_PASSWORD= in response body (w=0.95)

C. Detection Algorithms — Pseudocode

Algorithm 1 presents the core scoring procedure. Algorithm 2 presents the adaptive threshold optimisation.

Algorithm 1: HybriScan Category Scoring

```
Input:  url (string), response_body (string), pattern_banks (dict)
Output: score_vector (dict), composite_score (float)

FUNCTION ComputeVulnerabilityScore(url, response_body, pattern_banks):
  combined ← CONCAT(url, ' ', response_body)
  score_vector ← {}

  FOR category IN {admin, sqli, login, dir_listing, xss, sensitive}:
    raw_score ← 0.0
    max_score ← SUM(weight FOR (pattern, weight) IN pattern_banks[category])

    FOR (pattern, weight) IN pattern_banks[category]:
      matches ← COUNT(FINDALL(pattern, combined, flags=IGNORECASE))
      raw_score ← raw_score + (MIN(matches, 1) × weight)

    score_vector[category] ← MIN(1.0, raw_score / max_score)

  composite_score ← MAX(score_vector.values())
  RETURN score_vector, composite_score

FUNCTION Classify(composite_score, threshold T):
  IF composite_score >= T THEN RETURN vulnerable
  ELSE RETURN benign
```

Algorithm 2: Adaptive Threshold Optimisation

```
Input:  dataset D, initial_threshold T0=0.70, max_iter=30
        target_FPR=0.15, target_accuracy=0.75
        T_min=0.50, T_max=0.82
Output: optimised_threshold T, final_metrics M

FUNCTION OptimiseThreshold(D, T0, max_iter):
  T ← T0

  FOR iteration IN 1..max_iter:
    predictions ← [Classify(score(ep), T) FOR ep IN D]
    M ← Evaluate(predictions, D.ground_truth)

    IF M.FPR > target_FPR:
      delta ← 0.01   // fixed conservative step
```

```
      T ← MIN(T + delta, T_max)

    ELSE IF M.accuracy < target_accuracy:
      T ← MAX(T - 0.02, T_min)

    IF M.FPR <= target_FPR AND M.accuracy >= target_accuracy:
      BREAK  // Convergence criterion met

  RETURN T, M

// NOTE: This optimises T on the evaluation set (in-sample).
// Production deployment should use a separate held-out validation set.
```

D. Pattern Weight Rationale

Pattern weights reflect the discriminative specificity of each indicator. Patterns that match exclusively in the presence of a vulnerability (e.g., a specific MySQL error string) receive high weights (0.85-0.95). Patterns that match frequently in benign contexts (e.g., the substring 'admin' in a URL segment) receive low weights (0.20-0.35) to reduce false-positive contribution. Hard-negative discrimination depends critically on correctly weighting these low-specificity patterns. Future work will derive weights empirically using a logistic regression trained on a labeled dataset, replacing manual assignment.

Experimental Setup

A. Dataset

The evaluation dataset contains 27 endpoints targeting a synthetic host (target-976.local). Table II presents the complete dataset composition and per-category outcomes.

TABLE II. Dataset Composition and Per-Category Detection Outcomes

Category	N	Vulnerable	Benign	TP	FP	TN	FN	Category Recall
Admin Panel	4	4	0	3	—	—	1	75.0%
SQL Injection	3	3	0	1	—	—	2	33.3%
Login Page	3	3	0	1	—	—	2	33.3%
Directory Listing	3	3	0	3	—	—	0	100.0%
Hard Negative	6	0	6	—	1	5	—	N/A (FPR: 16.7%)
Safe Endpoint	8	0	8	—	0	8	—	N/A (FPR: 0.0%)
TOTAL	27	13	14	8	1	13	5	—

Note: FP/TN columns are not applicable to positive-only categories; TP/FN are not applicable to negative-only categories. Overall FP=1, FN=5, TP=8, TN=13.

B. Scanner Parameters

HybriScan was configured with: initial threshold $T_0 = 0.70$; maximum optimisation iterations = 30; FPR target = 0.15; accuracy target = 0.75; threshold bounds [0.50, 0.82]. The final optimised threshold was $T = 0.794$. Pattern weight coefficients were manually assigned based on domain expertise and are fixed across all experiments.

C. Baseline Configuration

The baseline wordlist scanner used the same path wordlist as HybriScan. It classified an endpoint as vulnerable if and only if its URL path matched a term in the admin, login, directory, or SQL-injectable path lists. No response content was examined. Both scanners processed the identical 27-endpoint dataset.

Results and Analysis

A. Confusion Matrices

TABLE III. Baseline Scanner Confusion Matrix (N=27)

	Pred: Vulnerable	Pred: Benign	Total
Act: Vulnerable	TP = 12	FN = 1	13
Act: Benign	FP = 14	TN = 0	14
Total	26	1	27

TABLE IV. HybriScan Confusion Matrix (N=27)

	Pred: Vulnerable	Pred: Benign	Total
Act: Vulnerable	TP = 8	FN = 5	13
Act: Benign	FP = 1	TN = 13	14
Total	9	18	27

B. Aggregate Performance Metrics

TABLE V. Performance Metric Comparison: Baseline vs. HybriScan

Metric	Baseline Scanner	HybriScan	Δ (Change)
Accuracy	44.44%	77.78%	+33.34 pp
Precision	46.15%	88.89%	+42.74 pp
Recall (TPR)	92.31%	61.54%	−30.77 pp
F1 Score	0.6154	0.7273	+0.1119
False Positive Rate	100.00%	7.14%	−92.86 pp
True Positives	12	8	−4
False Positives	14	1	−13
True Negatives	0	13	+13
False Negatives	1	5	−4 (i.e., +4 FNs)

The dominant finding is the elimination of 13 of 14 false positives (93% reduction), achieved by the response-content scoring engine. The cost is a reduction in recall from 92.31% to 61.54%, corresponding to 5 missed detections (false negatives). The F1 score improvement from 0.6154 to 0.7273 confirms that the precision gain outweighs the recall loss in the harmonic mean, indicating a net detection quality improvement.

C. Per-Category Analysis

Directory listing detection achieved perfect recall (3/3 TP, 0 FN). All three directory-listing endpoints returned composite scores at or above 0.82, driven by the high-weight directory index patterns in the response body. This category benefits from highly distinctive HTML markers (Index of /, Parent Directory) that do not appear in benign content.

Admin panel detection achieved 75% recall (3/3 TP, 1 FN). The missed detection, /admin?src=bookmark, scored 0.7035 — below the 0.794 threshold — despite individual category scores of 0.8835 (admin) and 0.8344 (login). This near-miss arises because the composite score uses maximum pooling across categories rather than a weighted average. A blended aggregation strategy would have classified this endpoint correctly.

SQL injection detection achieved the weakest performance (33% recall; 1/3 TP, 2 FN). The endpoint /api/v1/record?search='+UNION+SELECT scored 1.0 on the SQL category and was correctly flagged. The endpoint /catalog?q=1+OR+1=1 scored 0.2608, and /article?search='+UNION+SELECT scored 0.0. The latter score of 0.0 indicates that the response body generator failed to produce SQL-indicative content for this endpoint, exposing a gap in the simulation module rather than in the detection engine.

Login detection achieved 33% recall (1/3 TP, 2 FN). The near-miss endpoints /session/new (score = 0.7533) and /account/signin (score = 0.7421) both fall below 0.794 by margins of 0.041 and 0.053 respectively. These cases motivate per-category threshold configuration as a priority future enhancement.

Hard-negative performance: 5 of 6 hard negatives correctly classified as benign (83% TN). The single false positive, /forum/thread/admin-tips, scored 0.3558, but was classified as positive due to an admin category score of 0.5358

triggered by the URL substring 'admin-tips'. This is a URL-level false positive that response-body evidence could have overridden with a richer content signal.

D. Threshold Sensitivity

Table VI presents hypothetical performance at alternative threshold values, computed by reclassifying all 27 endpoints against their observed scores.

TABLE VI. Performance at Alternative Classification Thresholds (N=27)

Threshold T	Accuracy	Precision	Recall	F1	FPR	Notes
0.700	66.67%	68.75%	84.62%	0.7586	42.86%	High FP — 6 FPs
0.750	74.07%	80.00%	61.54%	0.6957	14.29%	Similar to final
0.794 (selected)	77.78%	88.89%	61.54%	0.7273	7.14%	Chosen operating point
0.820	81.48%	100.00%	53.85%	0.7000	0.00%	Zero FP, higher FN
0.850	74.07%	100.00%	38.46%	0.5556	0.00%	Excessively conservative

Table VI reveals a clear precision-recall trade-off across the threshold range. At T = 0.82, precision reaches 100% (zero false positives) at the cost of recall dropping to 53.85%. At T = 0.70, recall improves to 84.62% but FPR spikes to 42.86%. The selected threshold of 0.794 represents a balanced operating point. The absence of a ROC curve (which would plot this trade-off continuously) is a limitation of the current evaluation, addressed in future work.

Discussion

A. Interpretation of False Positive Reduction

The reduction of false positives from 14 to 1 is the primary operationally significant finding. In a real penetration testing engagement, each false positive requires manual verification. A scanner with 100% FPR on a 14-endpoint negative set provides no filtering value relative to manual enumeration. HybriScan's response-content scoring layer provides meaningful disambiguation, reducing the analyst's false-positive verification burden by 93% on this dataset.

B. The Precision-Recall Trade-off and Its Operational Implications

The recall reduction from 92.31% to 61.54% is a real cost. In security contexts, missed vulnerabilities (false negatives) can be more damaging than false alarms. Practitioners should therefore configure HybriScan's threshold based on the acceptable risk profile of the engagement: lower thresholds maximise recall at the cost of precision; higher thresholds maximise precision at the cost of recall. We recommend T = 0.75 as a conservative default for engagements where missed vulnerabilities are unacceptable, and T = 0.82 for noise-minimising assessments.

C. SQL Injection as the Weakest Category

The 33% recall for SQL injection is attributable to two distinct failure modes. First, the response simulation module did not generate SQL-indicative content for all SQL-injection category endpoints. This is a data generation bug, not a detection engine weakness. Second, the SQL scoring engine relies on error message leakage — a detection strategy that fails against applications configured to suppress database error output (which is a security best practice). This motivates the addition of active payload injection as a detection mechanism, wherein the scanner sends a modified request with an SQL payload and compares the response to a baseline to detect differential behaviour.

D. Hard-Negative Results and Their Significance

The 83% TN rate on hard negatives (5/6 correctly classified) is a practically significant result. False positives generated by keyword-matching systems on benign security-discussion pages (such as blog posts about admin security or SQL injection tutorials) are a known frustration for security teams. HybriScan's weighted response-body scoring correctly filters 5 of 6 such endpoints. The single FP arises from URL-level keyword matching without a corresponding response-body negative signal — a fixable issue through response-body down-weighting.

Ethical Considerations

All experimental scanning in this study was conducted exclusively against synthetically generated endpoint data on an isolated local network without targeting any production system or third-party infrastructure. The HybriScan tool described in this paper is designed for authorised security assessment exclusively. Deployment against any system without explicit written authorisation from the system owner violates applicable computer misuse legislation including but not limited to the Computer Fraud and Abuse Act (United States), the Computer Misuse Act 1990 (United Kingdom), and the Information Technology Act 2000 (India). The authors disclaim all responsibility for misuse.

The planned future evaluation on DVWA, WebGoat, and Mutillidae will be conducted exclusively on locally hosted instances within an isolated private network environment. No public or production systems will be used. Results will be collected and reported in compliance with responsible disclosure principles.

Limitations

- Dataset Size: The 27-endpoint evaluation set is insufficient for statistical significance testing. Results should be interpreted as proof-of-concept rather than production-grade performance estimates.
- Synthetic Responses: All response bodies are simulated. Real web servers produce diverse, framework-dependent responses that may activate or suppress patterns in ways not captured by the simulation.
- In-Sample Threshold Tuning: The threshold is optimised on the same data used for evaluation, constituting in-sample metric reporting. A held-out test set is required for out-of-sample performance estimation.
- No JavaScript Rendering: The HTTP client does not execute JavaScript. Single-page applications (SPAs) that dynamically render content client-side are not fully scannable; only server-delivered HTML is analysed.
- Manual Pattern Weights: Weight coefficients are manually assigned without empirical derivation. They may not generalise to web frameworks or content types not represented in the evaluation dataset. Future work will derive weights using supervised learning on a labelled corpus.
- Single Global Threshold: A single classification threshold is applied across all six vulnerability categories. Per-category thresholds would likely improve recall without increasing FPR.

No Authenticated Scanning: HybriScan operates exclusively in unauthenticated mode. Application areas requiring session-based login are not scanned, potentially missing vulnerabilities accessible only to authenticated users.

Conclusion

This paper presented HybriScan, a category-aware, heuristic web vulnerability scanner that augments traditional wordlist-based path enumeration with a weighted, multi-category response-content scoring engine. Evaluation on a 27-endpoint controlled dataset demonstrated that HybriScan achieves an accuracy of 77.78%, a precision of 88.89%, and a false-positive rate of 7.14%, compared to the pure-wordlist baseline's 44.44% accuracy and 100% FPR. The F1 score improves from 0.6154 to 0.7273, confirming a net improvement in the precision-recall balance.

The per-category analysis reveals that directory listing detection is the most reliable category (100% recall), while SQL injection detection is the weakest (33% recall), primarily due to limitations in the response simulation module and the error-message-leakage dependency of the SQL pattern bank. Five of six hard-negative endpoints are correctly classified as benign, demonstrating that weighted response-body scoring provides meaningful discrimination against keyword-rich benign content.

HybriScan represents a transparent, reproducible, and computationally lightweight baseline for response-aware web vulnerability detection. Its architecture is explicitly designed for extensibility: the pattern banks, weight coefficients, and threshold mechanism can be replaced with data-driven components as the evaluation dataset scales to production size.

Future Work

- Real-World Validation: Test HybriScan against DVWA (all security levels), OWASP WebGoat, and Mutillidae II hosted in local Docker containers. Collect 380+ endpoint dataset with human-verified ground-truth labels. Report out-of-sample metrics with 95% confidence intervals.
- Comparison with Production Tools: Benchmark HybriScan against Nikto v2.1.6, Gobuster v3.6, Dirsearch v0.4.3, and OWASP ZAP 2.15 on the same DVWA/WebGoat targets. Produce side-by-side metric tables.

- ROC/AUC Analysis: Implement threshold sweep from 0.00 to 1.00 in 0.01 steps. Plot ROC curve. Compute AUC. Report AUC as the primary threshold-independent performance metric.
- Per-Category Threshold Optimisation: Replace the global threshold with six independent category thresholds, each optimised separately on a category-stratified validation split.
- Differential Payload Analysis: Extend the existing opt-in payload testing module (payload_tester.py) beyond error-indicator and reflection detection to support differential timing analysis and multi-parameter interaction testing across a broader payload corpus.
- Machine Learning Integration: Extract the six-dimensional category score vector as a feature vector. Train a scikit-learn LogisticRegression or GradientBoostingClassifier on a labeled 380+ endpoint dataset. Compare ML-augmented performance against the heuristic-only version.
- Authenticated Scan Support: Extend the HTTP acquisition layer to support session-cookie injection and form-based login to enable scanning of authenticated application areas currently inaccessible to the unauthenticated scanner.
- JavaScript Rendering: Integrate a headless browser (e.g., Playwright) to render JavaScript-heavy Single-Page Applications before pattern analysis, extending coverage to dynamically generated content not present in the raw HTTP response.
- Explainability Interface: Develop a human-readable explanation module that articulates, for each flagged endpoint, which specific patterns triggered the classification and their individual score contributions.

References

[1] OWASP Foundation, "OWASP Top Ten 2021," Open Web Application Security Project, 2021. [Online]. Available: https://owasp.org/Top10/

[2] C. Sullo and D. Lodge, "Nikto: Web Server Scanner," 2002. [Online]. Available: https://cirt.net/Nikto2

[3] OJ Reyes, "Gobuster: Directory/File & DNS Busting Tool," GitHub, 2019. Available: https://github.com/OJ/gobuster

[4] M. Peretti, "Dirsearch: Web Path Discovery Tool," GitHub, 2018. Available: https://github.com/maurosoria/dirsearch

[5] I. Medeiros, N. F. Neves, and M. Correia, "Detecting and Removing Web Application Vulnerabilities with Static Analysis and Data Mining," IEEE Trans. Reliability, vol. 65, no. 1, pp. 54–69, Mar. 2016.

[6] A. Doupé, M. Cova, and G. Vigna, "Why Johnny Can't Pentest: An Analysis of Black-Box Web Vulnerability Scanners," in Proc. 7th Int. Conf. DIMVA, 2010, pp. 111–131.

[7] J. Bau, E. Bursztein, D. Gupta, and J. Mitchell, "State of the Art: Automated Black-Box Web Application Vulnerability Testing," in Proc. IEEE Symp. Security and Privacy (S&P), 2010, pp. 332–345.

[8] G. Pellegrino, C. Tschürtz, E. Bodden, and C. Rossow, "jÄk: Using Dynamic Analysis to Crawl and Test Modern Web Applications," in Proc. RAID, 2015, pp. 295–316.

[9] M. Paramitha and K. Kusrini, "Web Application Vulnerability Detection using Machine Learning," Int. J. Advanced Computer Science and Applications, vol. 12, no. 5, 2021.

[10] C. Shu, H. Chen, and Q. Li, "Identifying Web Vulnerabilities Using Transformer-Based Sequence Models," IEEE Transactions on Reliability, 2022.

[11] Y. Li, V. Toffalini, J. Losiouk, and M. Conti, "WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning," IEEE Access, vol. 8, 2020.

[12] PortSwigger Ltd., "Burp Suite Professional Documentation," 2024. [Online]. Available: https://portswigger.net/burp/documentation

[13] NIST, "National Vulnerability Database," 2024. [Online]. Available: https://nvd.nist.gov/

[14] OWASP Foundation, "WebGoat: Deliberately Insecure Web Application," 2023. Available: https://owasp.org/www-project-webgoat/

[15] D. C. Ramos, "DVWA: Damn Vulnerable Web Application," GitHub, 2023. Available: https://github.com/digininja/DVWA